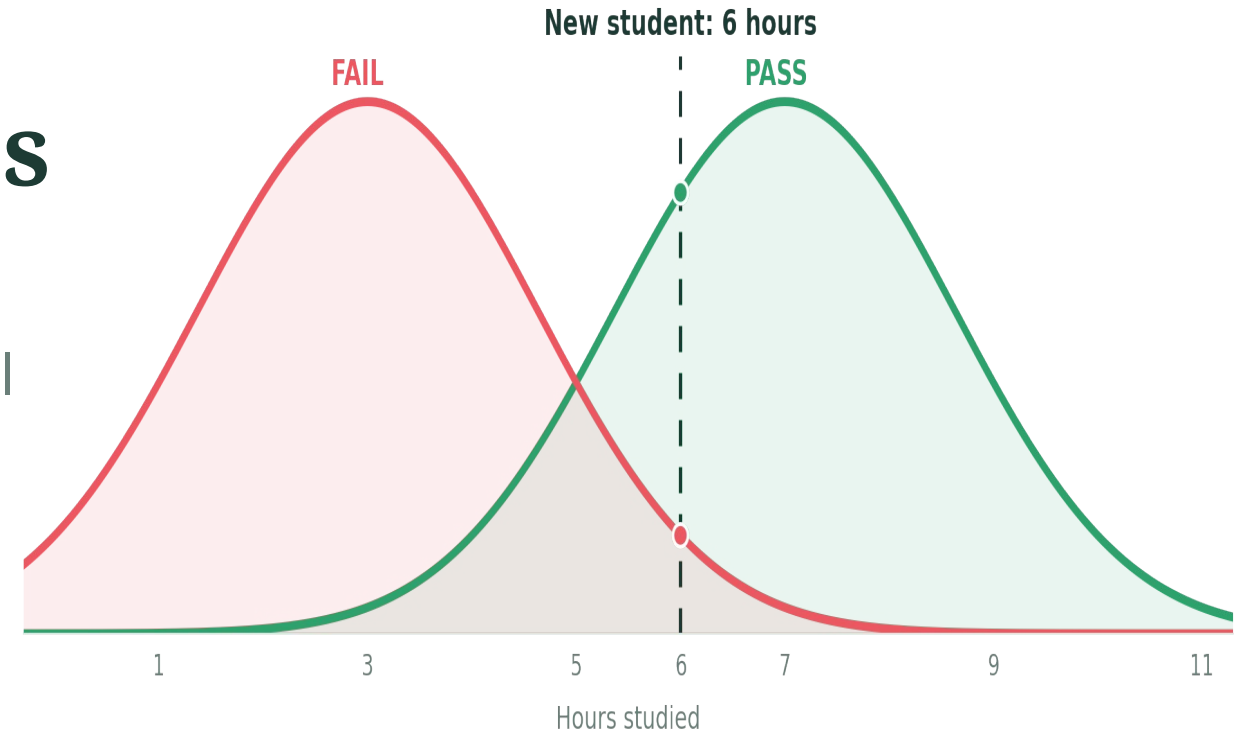
Naive Bayes with numerical values

From counting categories to fitting a bell curve



One worked example • Step by step • Simple Python

Start with something you already know

For categories, we count within each class.

FROM YOUR PREVIOUS LESSON

Coaching = Yes

Among 6 passing students, 4 attended coaching.

P(Yes | Pass) = 4 ÷ 6

= 0.667

That method works because “Yes” is a countable category.

Now replace the category with a number

We want to predict Pass or Fail from study hours.

Student	Hours studied	Result
A	5	Pass
B	7	Pass
C	9	Pass
D	1	Fail
E	3	Fail
F	5	Fail

New student

Studied 6 hours

Will the student Pass or Fail?

Why exact counting fails for numerical data

No student in this small dataset studied exactly 6 hours.

If we use counting...

$P(\text{Hours} = 6 \mid \text{Pass})$

$= 0 \div 3 = 0$

But 6 is close to...

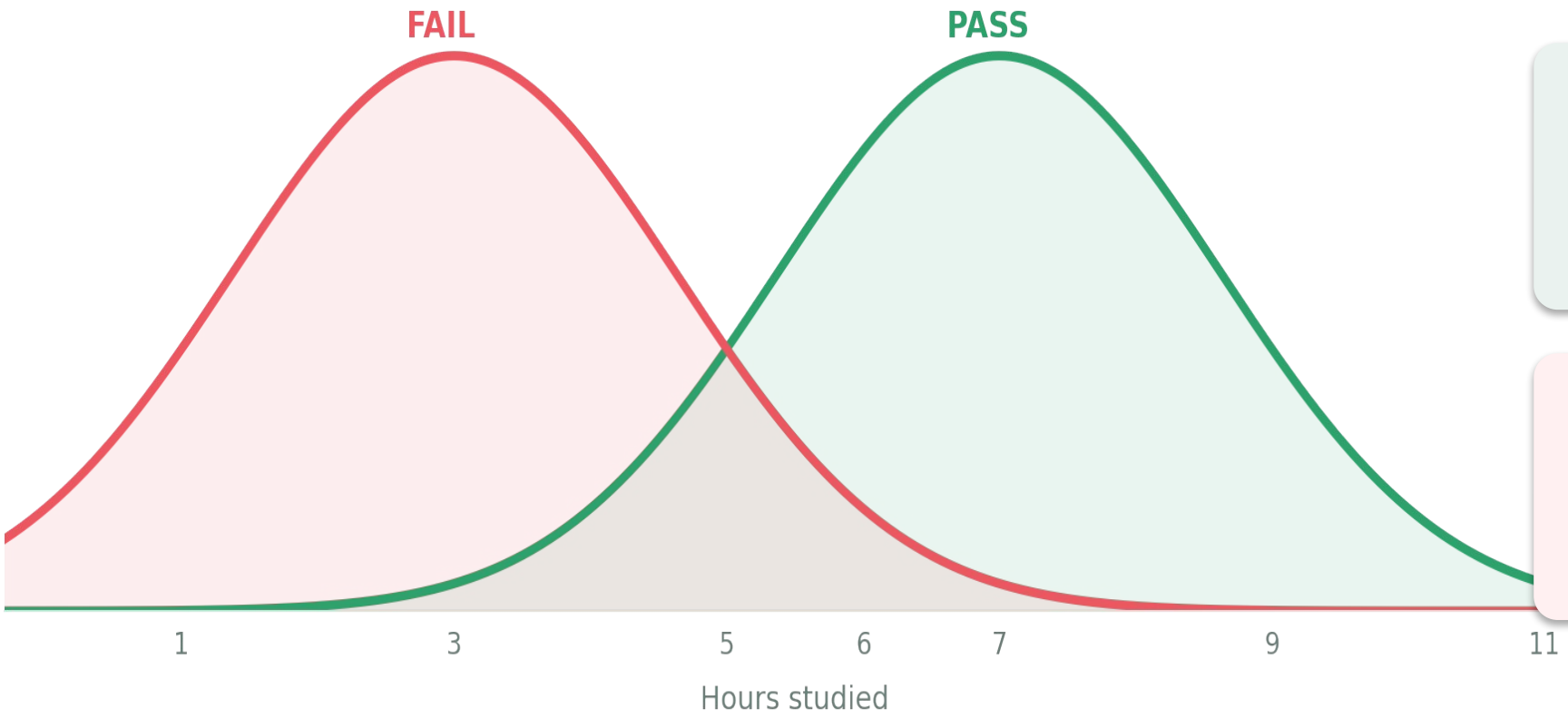
5 and 7 hours

So a likelihood of zero would be misleading.

We need a method that works even when a specific number was never seen.

The Gaussian idea: fit a bell curve

Each class gets its own distribution of study hours.



Pass curve

Centred near 7

Fail curve

Centred near 3

A new value can fit one curve better than the other.

Step 1: calculate the mean for each class

Mean = total study hours ÷ number of students.

PASS

5, 7, 9 hours

Mean = (5 + 7 + 9) ÷ 3

= 7 hours

FAIL

1, 3, 5 hours

Mean = (1 + 3 + 5) ÷ 3

= 3 hours

The mean tells us where each bell curve is centred.

Step 2: calculate how spread out the values are

This spread is measured with variance.

Pass: 5, 7, 9 → mean 7

Distances from mean: −2, 0, +2

Square and average:

(4 + 0 + 4) ÷ 3 = 2.67

Variance

2.67

Standard deviation ≈ 1.63

Fail: 1, 3, 5 → mean 3 → variance is also 2.67.

What do mean and variance change?

Mean moves the curve. Variance controls its width.

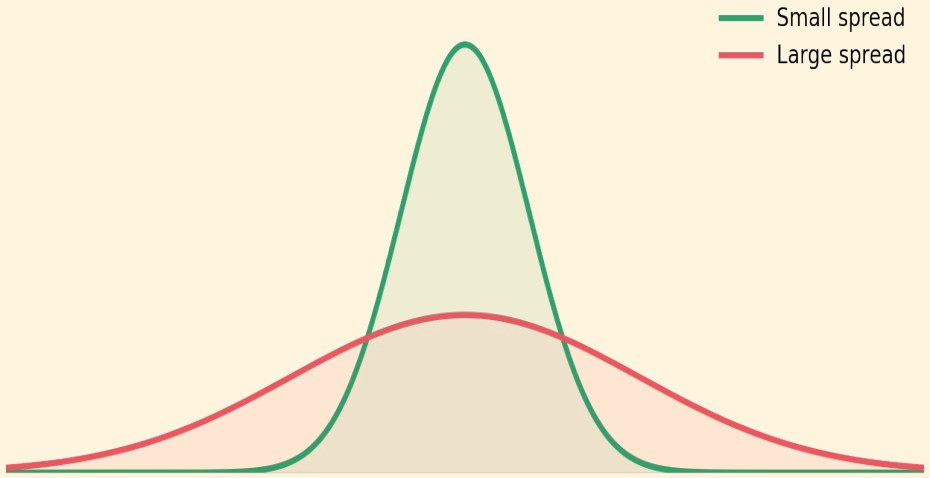
MEAN (centre)

Pass mean = 7

Fail mean = 3

Different means → curves in different places.

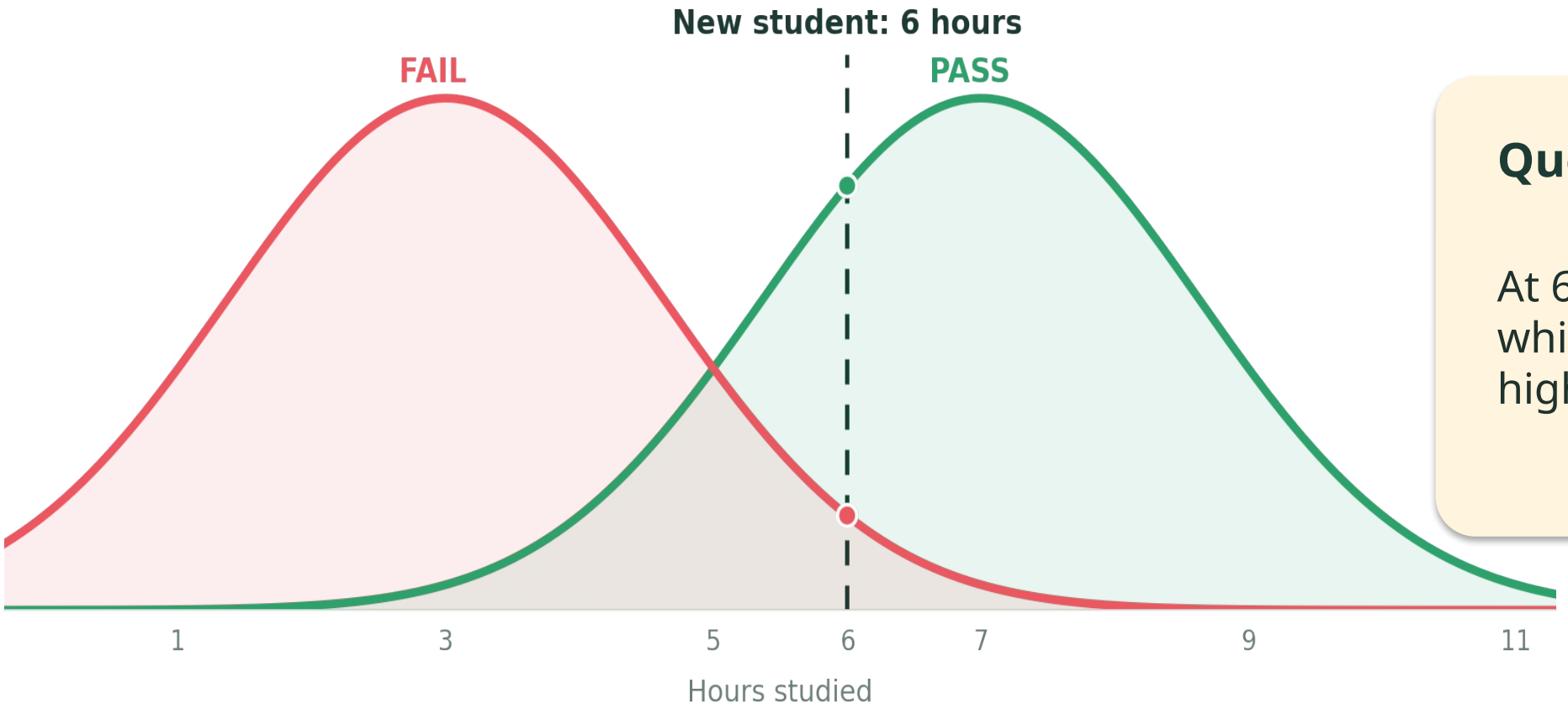
VARIANCE (spread)



Higher variance → wider, flatter curve.

Step 3: see where 6 hours fits

Place the new student on both curves.



Question

At 6 hours, which curve is higher?

Higher curve = stronger fit for that class.

Step 4: use the bell-curve formula

The formula converts any numerical value into a density for each class.

Gaussian density = $(1 \div \sqrt{(2\pi \times \text{variance})}) \times \exp(-(x - \text{mean})^2 \div (2 \times \text{variance}))$

WHAT WE PUT INTO IT

x	**6 hours**	Pass mean	**7 hours**
Fail mean	**3 hours**	Variance	**2.67 each**

Step 5: compare the two likelihood densities

For the new student, x = 6 hours.

PASS: mean 7, variance 2.67

Density at 6 hours

0.203

FAIL: mean 3, variance 2.67

Density at 6 hours

0.045

These are density values — not yet the final Pass/Fail probabilities.

Step 6: multiply by the class prior

Three students passed and three failed, so both priors are 0.5.

Class	Prior	Density	Score
Pass	0.5	≈0.203	**≈0.1013**
Fail	0.5	≈0.045	**≈0.0226**

$$\text{Score}(\text{Class}) = \text{Prior}(\text{Class}) \times \text{Density}(\text{Hours} = 6 \mid \text{Class})$$

Step 7: normalize to get the final prediction

Divide each score by the sum of both scores.

P(Pass | 6 hours) ≈ 0.1013 ÷ (0.1013 + 0.0226)

≈ 81.8% Pass

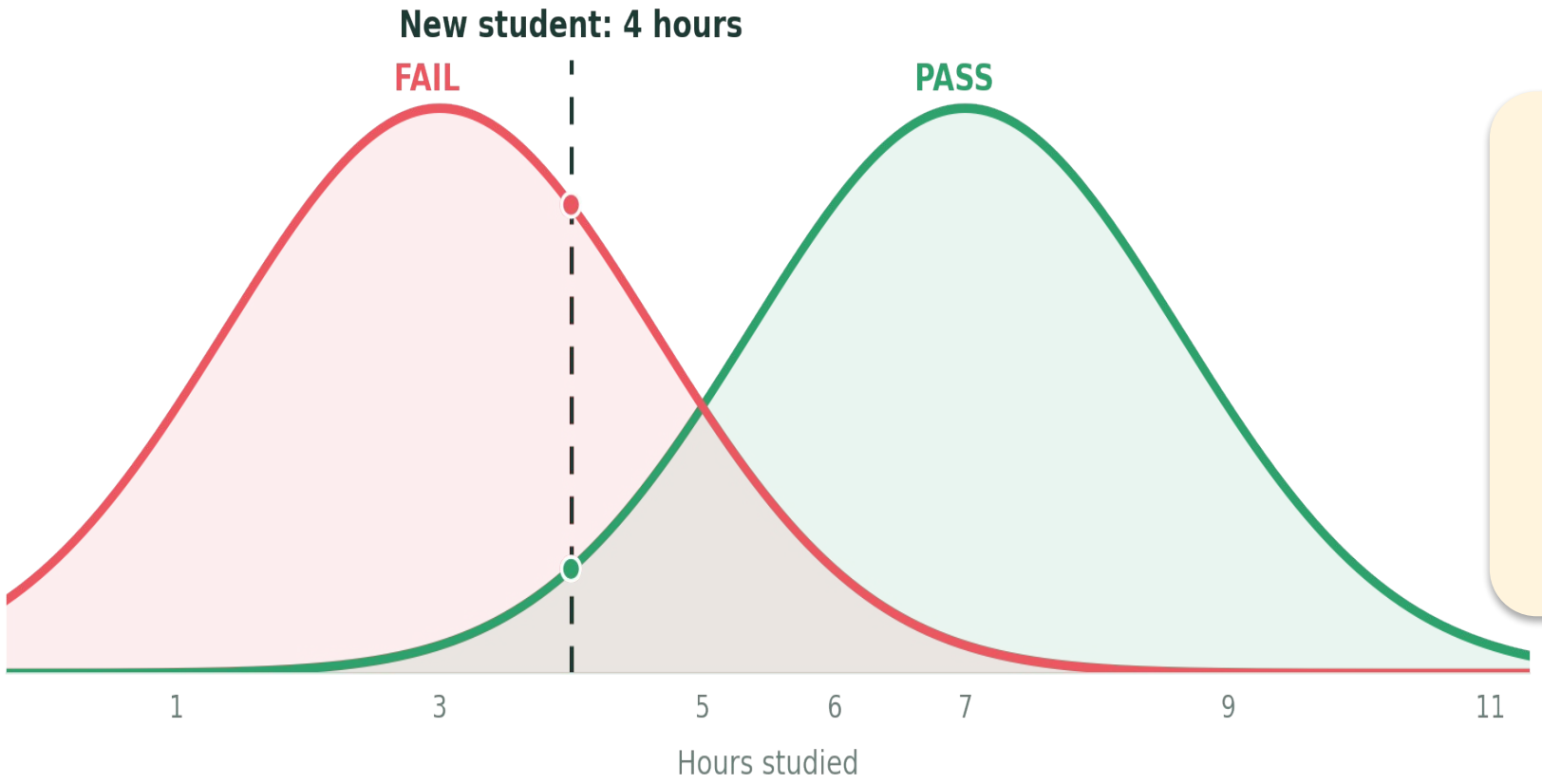


Pass: 81.8% **Fail: 18.2%**

Prediction: Pass — because the Pass posterior is higher.

Your turn: what about 4 hours?

Same training data. Do not change the bell curves.



Class question

More likely to:

PASS or FAIL?

Think first: which curve is higher at x = 4?

Check your answer: 4 hours → Fail

The curve near 3 hours has the higher density at x = 4.

Pass likelihood density

0.045

Fail likelihood density

0.203

After priors and normalization:



Pass 18.2% **Fail 81.8%**

What if we have more than one numerical feature?

Example: study hours and mock test score.

New student: Hours = 6 Mock score = 75

For the Pass class

Prior(Pass) × density(6 | Pass)
× density(75 | Pass)

For the Fail class

Prior(Fail) × density(6 | Fail)
× density(75 | Fail)

“Naive” assumption: features are treated as independent within a class.

Connect this to the Wine dataset

Wine has 178 samples, 13 numerical features and 3 classes.

Alcohol content: mean and variance within each Wine class

Wine class	Mean alcohol	Variance
Class 0	13.74	0.21
Class 1	12.28	0.29
Class 2	13.15	0.28

GaussianNB learns one mean and variance per feature, per class — not only for alcohol.

Python: the small study-hours example

Use GaussianNB to reproduce our worked prediction.

```
import numpy as np
from sklearn.naive_bayes import GaussianNB

X = np.array([[5], [7], [9], [1], [3], [5]])
y = ['Pass', 'Pass', 'Pass', 'Fail', 'Fail', 'Fail']

model = GaussianNB()
model.fit(X, y)

print(model.predict([[6]]))
print(model.predict_proba([[6]]).round(3))
```

Expected output

```
['Pass']

[[0.182, 0.818]]
```

Class order is alphabetical: Fail, then Pass.

See exactly what the trained model learned

The attributes theta_ and var_ give the class statistics.

```
print(model.classes_)
print(model.theta_)
print(model.var_)
```

Output (approximately)

```
['Fail' 'Pass']
[[3.], [7.]]
[[2.667], [2.667]]
```

theta_ = mean | var_ = variance

Python: train GaussianNB on Wine

The same code now works with 13 numerical features and 3 classes.

```
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

X, y = load_wine(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

model = GaussianNB()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
print('Accuracy:', accuracy_score(y_test, y_pred))
```

CategoricalNB vs GaussianNB

The Bayes rule does not change; the likelihood calculation does.

Concept	CategoricalNB	GaussianNB
Feature type	Yes / No / Late	Hours / Age / Alcohol
Model learns	Counts per class	Mean and variance per class
Likelihood	Smoothed frequency	Bell-curve density
Bayes calculation	Prior × likelihoods	Prior × densities

For GaussianNB, “likelihood” is a density, not a direct probability at an exact number.

Check understanding

Ask students these four questions before finishing.

01 **Why is counting “exactly 6 hours” not useful?**

02 **What does GaussianNB learn for each class?**

03 **What is the difference between density and posterior?**

04 **If Fail density is higher at 4 hours, which class wins?**

TAKEAWAY

Same Bayes idea.
Different way to measure evidence.

Categorical

Count feature values

Gaussian

Learn mean + variance

Both

Multiply by prior, normalize

Next: open the Wine notebook and inspect model.theta_, model.var_, and predict_proba().